Sprint 1 Planning Document — Capstone Design Project Management

Sponsor: Dr. Monica Anderson | Team: Cole Kelley, Cole Munn, Gates Arnold, Jackson Burke, Thomas Hicks

Planned Sprint Start: January 29, 2026

Planned Sprint End: February 24, 2026

1. Sprint Goal

Establish the project foundation: select tech stack and architecture, define core data models, expose CRUD via a REST API, and stand up a basic Vue UI capable of listing records—so we can begin flowing sponsor/project/student data into the system in subsequent sprints.

2. Scope (Planned for Sprint 1)

- Architecture & Tech Stack: Vue.js frontend, Django REST API, PostgreSQL database; all services containerized with Docker (separate containers for Vue, Django, Postgres).
- Data Model Definition (Django): Project, Sponsor, Student, Assignment, Preference, Attachment, with enums/choices where specified.
- API Endpoints: CRUD for all core models with basic validation and error handling.
- Frontend: Landing pages to list key entities (e.g., Projects, Students, Sponsors) via API integration.
- Dev Environment: Docker Compose scripts, environment variables, basic README with setup and run instructions.

3. Non-Goals / Deferred Items (Out of Scope for Sprint 1)

- Authentication & Role-Based Routing (Admin/Student/Sponsor); OAuth integration pending UA identity migration decisions.
- Bulk Email/Notification System (templates, audiences).
- Manual Assignment UI (admin) and rich preferences explorer (search/filter).
- File Upload storage pipeline (beyond attachment links).
- Algorithmic matching of students to projects.
- Cloud deployment and VM snapshot/backup configuration.

4. User Stories Targeted in Sprint 1

1. As a developer, I need a containerized environment (Vue, Django, Postgres) so that the app runs consistently across machines.

2. As a developer, I need core data models that reflect sponsor requirements so later features (ranking, assignments, attachments) are unblocked.
3. As a developer, I need RESTful CRUD endpoints for each model so the frontend can integrate and test flows early.
4. As a student user, I can view a list of projects so I can understand what is available (read-only for now).

5. Acceptance Criteria / Definition of Done (DoD)

- Docker Compose starts Vue, Django, and Postgres successfully with documented steps.
- Django migrations run cleanly; tables exist for all core models.
- CRUD endpoints operational for Project, Sponsor, Student, Assignment, Preference, Attachment (happy paths tested).
- Vue renders at least one list view per major entity, reading from the live API.
- Basic README covers setup, environment variables, Docker, and API endpoint map.

6. Deliverables

- Dockerized repo with compose file(s) and service definitions.
- Django app with models, serializers, viewsets, and URLs for all core resources.
- Vue app with landing pages listing Projects (and other entities as time permits).
- Seed/fixture data for demo purposes.

7. Risks & Assumptions

- UA identity migration may impact OAuth strategy and attribute mapping; temporary email-based role routing may be used until finalized.
- Attachment handling initially limited to links; full file storage requires policy, scanning, and quotas.
- Manual assignment will precede any automated matching; ensure schema supports semester/year and team size constraints later.
- Cloud deployment (VM/GCP) deferred; local Docker used during Sprint 1.

8. Dependencies

- Access to sponsor-provided project/sponsor examples for seed data
- Decision placeholder for OAuth (pending university guidance)

Appendix A — Data Model Overview (Planned)

- Project: id, project_name, project_description, sponsor_id, project_website (nullable), year_began (int), status (enum)
- Sponsor: id, name, email, phone_number (nullable), organization (nullable)

- Student: id, name, email
- Assignment: id, project_id (fk), person_id (fk), person_type (enum: sponsor|student), semester (enum), year (int)
- Preference: id, student_id (fk), project_id (fk), rank_value (int)
- Attachment: id, project_id (fk), file_url